

Assignment 3: Time Series Analysis

Classifying Cardiac Arrhythmias: A Comparative Analysis

Weight	25% of final grade
Deadline	31.05.2026 (23:59 CET)
Deliverables	Jupyter Notebook + Written Report (PDF)
Dataset	MIT-BIH Arrhythmia Database (PhysioNet)

Overview

In this assignment you will train, tune, and compare two temporal modelling approaches — Recurrent Neural Networks (RNN) and Long Short-Term Memory networks (LSTM) — on one of the most widely studied biomedical time series benchmarks: the MIT-BIH Arrhythmia Database. Your primary goal is to classify individual heartbeats into one of three clinically meaningful categories, then critically evaluate each method's strengths and limitations.

The assignment is structured around a single shared feature matrix: you will engineer 8 features per heartbeat that capture rhythm, morphology, and repolarisation. These same features feed every model, making the comparison between methods direct and interpretable. HMM, RNN, and LSTM consume the features differently — independently per beat, or across a sliding window of $T = 10$ consecutive beats — and understanding why this matters is a central learning objective.

Classification tasks account for approximately 65% of the total marks. This assignment is not a deep learning course: the RNN and LSTM are treated as sequence classifiers, and interpretability of results is expected alongside performance numbers.

Dataset

MIT-BIH Arrhythmia Database (PhysioNet)

The MIT-BIH Arrhythmia Database contains 48 half-hour excerpts of two-channel ambulatory ECG recordings from 47 subjects, digitised at 360 Hz with 11-bit resolution. Each beat has been manually annotated by cardiologists with a symbol indicating its type. The database is freely accessible via the

wfdb Python library, which downloads records directly from PhysioNet — no manual download required:

```

pip install wfdb

import wfdb
record = wfdb.rdrecord('100', pn_dir='mitdb') # downloads
automatically
ann     = wfdb.rdann('100', 'atr', pn_dir='mitdb')
signal = record.p_signal[:, 0]                # use channel 0 (MLII
lead)

```

You will work with the following 14 records, which provide a representative mix of normal rhythm and clinically significant arrhythmias:

```
100, 101, 105, 106, 108, 109, 111, 112, 115, 117, 119, 201, 213, 219
```

This subset yields approximately 35,000–40,000 labelled beats, sufficient for robust model training and evaluation.

Classification Target: AAMI 3-Class Scheme

The raw annotation files contain over 15 distinct beat symbols. You will map these to a simplified 3-class scheme based on the AAMI EC57 standard, which is the convention used in most published benchmark results on this dataset:

```

label_map = {
    # N - Normal
    'N': 'N', 'L': 'N', 'R': 'N', 'e': 'N', 'j': 'N',
    # S - Supraventricular ectopic
    'A': 'S', 'a': 'S', 'J': 'S', 'S': 'S',
    # V - Ventricular ectopic
    'V': 'V', 'E': 'V',
    # All other symbols: discard
}

```

Any beat whose symbol does not appear in `label_map` (e.g. pacemaker beats ‘P’, noise ‘~’, rhythm annotations) should be discarded. Document how many beats you discard and from which classes.

Class	Label	Typical examples	Expected frequency
N	Normal / bundle branch beats	Normal sinus, LBBB, RBBB, escape beats	~ 80–90% of all beats
S	Supraventricular ectopic	Atrial premature (A), nodal (J)	~ 2–5%

V	Ventricular ectopic	PVC (V), ventricular escape (E)	~ 5–10%
---	---------------------	---------------------------------	---------

Note the strong class imbalance: N beats dominate the dataset. You are expected to discuss how this affects your models and evaluation choices (e.g. why macro F1 is more informative than accuracy here).

R-Peak Locations

R-peak locations are provided directly in the annotation files and do not need to be detected algorithmically. The annotation object returned by wfdb gives you:

```
ann.sample # numpy array of R-peak sample indices (one per beat)
ann.symbol # numpy array of beat label strings, same length as
ann.sample
```

You should use `ann.sample` as the authoritative R-peak positions for all feature extraction. Do not apply an independent peak detector.

Feature Extraction

You will engineer **8 features per beat**. These same features are used by **all three models**. HMM receives one feature vector per beat; RNN and LSTM receive a sliding window of $T = 10$ consecutive beat vectors (shape: 10×8) with the label taken from the final beat in the window.

#	Feature	Extraction Window	Clinical Meaning
1	RR_current	Beat i	Current RR interval (ms) — primary rate indicator
2	RR_prev	Beat i-1	Previous RR interval — captures beat-to-beat variability
3	RR_ratio	Beat i	RR_current / RR_local_mean — relative rate change
4	RR_local_mean	Beats i-4 to i	5-beat rolling mean RR — local rhythm baseline
5	R_amplitude	± 5 samples	Signal amplitude at the R-peak
6	QRS_duration	Threshold crossing	Width of QRS above 50% of R amplitude
7	QRS_energy	± 20 samples	Sum of squared signal values around R-peak

8	ST_mean	+40 to +120 samples	Mean signal in ST segment — repolarisation indicator
---	---------	---------------------	--

Worked example — understanding RR features

Suppose the ECG signal is sampled at 360 Hz and wfdb reports the following R-peak sample indices for five consecutive beats in record 100:

```
ann.sample = [77, 370, 663, 947, 1237, ...]
```

These are sample-index positions of R-peaks — not times, not amplitudes, just positions in the signal array. For beat $i = 4$ (the 5th beat, 0-indexed), with R-peak at sample 1237:

- **RR_current** = $(1237 - 947) / 360 \times 1000 = 290 / 360 \times 1000 \approx 806$ ms. This is simply the time gap between this R-peak and the previous one, converted to milliseconds. A healthy resting heart rate of ~75 bpm corresponds to an RR interval of ~800 ms.
- **RR_prev** = $(947 - 663) / 360 \times 1000 \approx 789$ ms. The interval of the preceding beat. Comparing RR_current to RR_prev captures sudden rhythm changes: a premature ventricular contraction (V beat) will have a shorter RR_current followed by a longer compensatory pause.
- **RR_local_mean** = mean of the 5 intervals from beats $i-4$ through i = $\text{mean}(293, 293, 284, 290) \approx 800$ ms (in sample-difference terms, converted to ms). This rolling mean smooths out single outlier beats and acts as the local baseline rate.
- **RR_ratio** = $806 / 800 \approx 1.008$. A value close to 1.0 means this beat arrived on schedule. A value of 0.6 would indicate the beat arrived 40% earlier than the local average — a hallmark of an ectopic beat. This ratio normalises for patient-to-patient heart rate differences.

The remaining four features (R_amplitude, QRS_duration, QRS_energy, ST_mean) are all computed from a small window of signal samples centred on the R-peak at sample 1237 — see the Extraction Window column in the table above for the exact offsets.

Implementation notes:

- **RR intervals:** compute as the difference in sample indices between consecutive R-peaks, then convert to milliseconds by dividing by the sampling rate (360 Hz) and multiplying by 1000.
- **First beat edge case:** the first beat in a record has no preceding beat. Set RR_current, RR_prev, and RR_local_mean to the mean RR of the first 5 available beats of the record. Document this decision in your notebook.
- **QRS_duration:** find the first sample before and after the R-peak where the signal drops below 50% of R_amplitude. QRS_duration is the distance in samples between these two crossings.
- **Concatenate across records:** extract features independently per record, then concatenate into a single DataFrame before splitting. Retain a 'record_id' column for traceability but do not include it as a model feature.

The resulting feature matrix should have shape (n_beats, 8) where n_beats $\approx$ 35,000–40,000 across all 14 records.

Assignment Tasks

The assignment follows a single end-to-end pipeline. All team members should understand every stage, even if responsibilities are divided.

Task 1 — Data Loading & Exploration

- Load all 14 records using `wfdb`. For at least two records, plot a 5-second segment of the raw ECG signal with R-peak positions marked. Annotate visible beat types.
- Report the total number of beats per class (N, S, V) across all 14 records. Produce a bar chart of the class distribution. Is the dataset imbalanced? Discuss how imbalance might affect model training and which evaluation metrics are most appropriate.
- After computing the 8 features in Task 2, produce a correlation heatmap of the feature matrix. Identify and briefly discuss the most strongly correlated pairs.

Task 2 — Preprocessing & Feature Engineering

- Load annotations for all 14 records. Apply `label_map` to map raw symbols to N, S, or V. Discard beats with unmapped symbols and report the count discarded.
- Extract all 8 features per beat as specified in the Feature Extraction section. Handle the first-beat edge case and document your approach.
- **Scaling:** apply `StandardScaler` to the 8 features. Fit on the training set only and transform both train and test sets.
- **Train/test split:** apply a stratified 80/20 random split (`random_state=42`). Verify that class proportions are preserved in both splits and report the beat counts per class in each set.
- Construct the sliding window dataset for RNN and LSTM: for each beat at position $i \geq 10$, create a sample consisting of the feature vectors for beats $i-9$ through i (shape: 10×8) with label equal to the class of beat i . Apply the same 80/20 split.

Task 3 — RNN: Sequence Classification

- **Dataset:** use the sliding window dataset from Task 2 (shape per sample: 10×8). Each sample is a sequence of 10 consecutive beats; the label is the class of the final beat.
- Implement a vanilla RNN classifier in PyTorch (`torch.nn.RNN`) with one hidden layer and a Linear output layer for 3-class classification. Use cross-entropy loss and the Adam optimiser.
- Train for at least 20 epochs. Plot training and validation loss and accuracy curves per epoch.
- Produce a classification report and confusion matrix on the test set.
- Discuss the vanishing gradient problem in the context of this task. Why does a window of only 10 beats partially mitigate this issue, and why would longer sequences cause problems for a vanilla RNN?

3.5 — Extension: LSTM (+5 bonus points)

- Replace `torch.nn.RNN` with `torch.nn.LSTM` in your model, keeping all other hyperparameters (hidden size, learning rate, epochs) identical. Note: `nn.LSTM` returns

`(output, (h_n, c_n))` instead of `(output, h_n)` — that is the only structural change required in the forward pass.

- Produce a classification report and plot training curves. Compare macro F1 and per-class recall (especially S and V) against your vanilla RNN. Did performance change? Briefly explain why the LSTM's gating mechanism might help or make no difference at this sequence length.

Task 4 — Model Comparison & Evaluation

- Produce a summary comparison table with the following columns: Model, Accuracy, Macro F1, Weighted F1.
- Discuss which model you would deploy in a real-time cardiac monitor and why, considering classification performance, latency, interpretability, and the cost of false negatives on ventricular beats.

Task 5 — Written Report

In addition to your Jupyter Notebook, submit a written report of 1,500–2,500 words addressing:

- A description of the MIT-BIH dataset, the feature engineering decisions you made, and how you handled class imbalance.
- An analysis of the RNN model: its architecture choices, inductive biases, strengths, and limitations for beat-level arrhythmia classification. If you completed the LSTM extension, compare the two.
- Your evaluation results: which method performed best, and why do you think that is given the properties of the dataset?
- Reflection: how do class imbalance, sequence length, and the shared feature representation influence algorithm choice?
- One limitation of your analysis and how you would address it with more time or data (e.g. inter-patient generalisation, class-weighted loss, additional features).

Deliverables

Item	Description	Format
Jupyter Notebook	Fully executed with all outputs visible. Code must be clean and commented.	<code>.ipynb</code>
Written Report	1,500–2,500 words. Figures may be embedded. Reference list not included in word count.	<code>.pdf</code>

Submit both files as a single **.zip** archive named: `TeamXX_TimeSeries.zip`

Recommended Libraries

You may use the libraries listed below. List all libraries and versions used in a dedicated cell at the top of your notebook. AI coding tools may be used to assist with syntax, but analytical decisions and the written report must be your own. Indicate clearly if AI tools were used.

Tasks 1 & 2 — Data Loading, Exploration & Preprocessing	
wfdb	PhysioNet data access — loads records and annotations directly
numpy	Numerical operations, array slicing, feature computation
pandas	Tabular storage of per-beat feature vectors and labels
matplotlib / seaborn	ECG signal plots, class distribution, correlation heatmap
scipy.signal	find_peaks — R-peak verification; signal utilities
sklearn.preprocessing	StandardScaler, LabelEncoder
sklearn.model_selection	train_test_split with stratification
Tasks 4 & 5 — RNN / LSTM	
torch	Core PyTorch tensors and autograd engine
torch.nn	RNN, LSTM, Linear layers, CrossEntropyLoss
torch.utils.data	Dataset, DataLoader — sliding window batching over beat sequences
torch.optim	Adam optimiser
sklearn.metrics	classification_report, confusion_matrix
matplotlib	Training / validation loss and accuracy curves
Task 4 — Model Comparison	
sklearn.metrics	f1_score — per-class and macro F1 scoring
matplotlib	Confusion matrix and F1 comparison plots
pandas	Summary comparison table
	Measuring training and inference time

Note on frameworks: Tasks 4 and 5 use **PyTorch (torch)**. If you are new to PyTorch, focus on two things: the `Dataset` / `DataLoader` pattern for building sliding window batches, and the manual training loop (forward pass → loss → backward → optimiser step). These differ most from the sklearn API you are used to, which follows the familiar `.fit()` / `.predict()` sklearn-style API.

Grading Rubric

Total: 50 points (counts as 25% of the final grade). The 5-point LSTM extension is bonus and can compensate for marks lost elsewhere.

Section	Pts	Criteria
Task 1: Data Loading & Exploration	8	
Signal plot & dataset shape	3	ECG signal plotted for at least 2 records. Shape, sampling rate, and record count reported.
Class distribution	3	Beat counts per class (N/S/V) reported. Class imbalance identified and discussed.
Feature correlation heatmap	2	Heatmap of the 8 engineered features produced and discussed.
Task 2: Preprocessing & Feature Engineering	12	
R-peak loading & label mapping	4	wfdb annotations loaded correctly. Raw symbols mapped to N/S/V using the provided label_map. Discarded symbols documented.
Feature extraction	5	All 8 features computed correctly per beat. Edge cases handled (e.g. first beat has no RR_prev).
Scaling & train/test split	3	StandardScaler applied correctly (fit on train only). Stratified 80/20 split. Class balance checked in both sets.
Task 3: RNN — Sequence Classification	15	
Sliding window dataset	4	Dataset/DataLoader correctly built with T = 10 beats. Shape verified.
RNN training	5	Vanilla RNN trained with cross-entropy loss. Learning curves (loss + accuracy) plotted.
Classification report	3	Confusion matrix and per-class metrics on test set produced.
Vanishing gradient	3	Short-memory limitation of vanilla RNN discussed. Motivation for LSTM stated.
LSTM extension (bonus)	5	nn.RNN replaced with nn.LSTM. Classification report produced and macro F1 compared to vanilla RNN.
Task 4: Model Comparison & Evaluation	5	

Summary table	5	Accuracy, Macro F1, Weighted F1 tabulated. Best model justified with discussion.
Task 5: Written Report	10	
Dataset & preprocessing	3	MIT-BIH dataset and feature engineering decisions explained clearly.
Algorithm comparison	4	Assumptions and trade-offs of HMM, RNN, and LSTM discussed accurately.
Reflection & conclusion	3	Results interpreted; one limitation identified; improvement proposed.
TOTAL	100	

Notes & Academic Integrity

- All team members are expected to understand and be able to explain every part of the submission.
- You may use the libraries listed in the Recommended Libraries section. List all libraries and versions in your notebook.
- AI coding tools may be used to assist with syntax, but analytical decisions and the written report must be your own. Indicate clearly if AI tools were used.

Advice for Teams

Divide the pipeline by task (one member per task), but hold a joint review session before submission. A notebook that **runs end-to-end without errors is a minimum requirement for full marks.**